

Module 10) Rest Framework

Introduction to APIs

1. What is an API (Application Programming Interface)?

- An **API (Application Programming Interface)** is a set of rules, protocols, and tools that allows **different software applications to communicate with each other**.
- It acts as a **bridge** between two systems, enabling them to exchange data and use each other's functionalities without knowing how the other system is internally built.

. Types of APIs

1. **Web APIs** – Used over the internet (e.g., REST, SOAP)
2. **Operating System APIs** – Allow applications to interact with the OS
3. **Library APIs** – Provide functions and methods from programming libraries
4. **Database APIs** – Enable communication between applications and databases

Advantages of APIs

- **Reusability** – Developers can reuse existing services
- **Efficiency** – Faster development and reduced coding effort
- **Scalability** – Easy integration with new systems
- **Security** – Controlled access to data and services

2. Types of APIs: REST, SOAP.

Feature	REST	SOAP
Type	Architectural style	Protocol
Data Format	JSON, XML	XML only
Speed	Faster	Slower
Complexity	Simple	Complex
Security	Custom implementation	Built-in WS-Security
Usage	Web & Mobile Apps	Enterprise Systems

3. Why are APIs important in web development?

APIs (Application Programming Interfaces) play a very important role in modern web development. They act as a **bridge between different software applications**, allowing them to communicate and share data with each other efficiently.

Importance of APIs in Web Development

1. Enables Communication Between Systems

APIs allow different applications (frontend, backend, mobile apps, third-party services) to communicate with each other smoothly.

For example, a website can fetch data from a server using an API without directly accessing the database.

2. Separation of Frontend and Backend

APIs help separate the **user interface (frontend)** from the **business logic (backend)**.

- Frontend: HTML, CSS, JavaScript
- Backend: Django, Node.js, Java, etc.

This separation makes development faster and more manageable.

3. Reusability of Code

Once an API is created, it can be reused by:

- Web applications
- Mobile applications
- Desktop applications

This reduces duplication of work and saves development time.

4. Integration with Third-Party Services

APIs make it easy to integrate external services such as:

- Payment gateways (Paytm, Razorpay)
- Google Maps
- Social login (Google, Facebook)
- Email and SMS services

Without APIs, such integrations would be very difficult.

5. Scalability and Performance

APIs allow applications to scale efficiently.

- Backend services can be upgraded without affecting the frontend.
- Multiple clients can use the same API simultaneously.

1. Understanding project requirements.

Project requirements are detailed descriptions of features, functions, and constraints that a system must fulfill. They act as a blueprint for designing, developing, and testing the project.

Why Understanding Project Requirements Is Important

1. Defines Project Goals Clearly

Clear requirements help developers understand:

- Purpose of the project
- Target users
- Expected outcomes

This avoids confusion and ensures everyone works toward the same goal.

2. Helps in Proper Planning

Understanding requirements helps in:

- Estimating time and cost
- Choosing the right technologies
- Planning resources and milestones

Good planning reduces project risks.

3. Avoids Future Changes and Errors

Well-defined requirements reduce:

- Rework
- Misunderstandings
- Last-minute changes

This saves time and development cost.

2. Setting up the environment and installing necessary packages.

Setting up the development environment ensures the project runs smoothly and without errors. It includes installing the required programming language, framework, tools, and libraries needed for development.

Key steps:

- Install required software (language, framework, IDE)
- Create a virtual environment
- Install necessary packages using a package manager
- Configure environment variables
- Test the setup by running the project

A proper environment setup helps avoid compatibility issues and speeds up the development process.

1. What is Serialization?

Serialization is the process of converting complex data (such as objects or models) into a format that can be easily stored or transmitted, like JSON or XML. This allows data to be sent between a server and a client.

In web development (especially in APIs), serialization is used to convert database data into a readable format for frontend or mobile applications, and deserialization converts that data back into objects.

Example:

Converting a Django model object into JSON data for an API response.

Types of Serialization

1. **JSON Serialization** – Most commonly used in REST APIs
2. **XML Serialization** – Used in SOAP-based services
3. **Binary Serialization** – Used for faster data storage and transfer

3. Converting Django QuerySets to JSON.

Methods to Convert Django QuerySets to JSON

1. Using Django REST Framework (Recommended)

Django REST Framework (DRF) uses **serializers** to convert QuerySets into JSON.

- Model data → Serializer → JSON response
- Clean, secure, and scalable approach

Example:

- A model QuerySet is passed to a serializer
- Serializer returns JSON data

2. Using `values()` Method

The `values()` method converts QuerySets into dictionaries, which can be easily converted to JSON.

- Returns data as key–value pairs
- Simple but limited for complex data

3. Using Django's serializers Module

Django provides a built-in serializer to convert QuerySets into JSON format.

- Supports JSON, XML, and YAML
- Useful for basic serialization

4. Using serializers in Django REST Framework (DRF).

Serializers in Django REST Framework are used to convert Django models and QuerySets into **JSON format** and to convert JSON data back into Django objects. They also handle **data validation**.

DRF provides **Serializer** and **ModelSerializer**, with ModelSerializer being the most commonly used because it automatically maps model fields. Serializers make it easy to build clean, secure, and scalable REST APIs.

1. HTTP request methods (GET, POST, PUT, DELETE).

HTTP request methods define the type of action a client wants to perform on a server. They are commonly used in web applications and REST APIs.

- **GET:** Used to retrieve data from the server. It does not modify data.
Example: Fetching a list of users.
- **POST:** Used to send data to the server to create a new resource.
Example: Creating a new user or placing an order.
- **PUT:** Used to update an existing resource completely.
Example: Updating user profile details.
- **DELETE:** Used to remove a resource from the server.
Example: Deleting a user record.

These methods help perform **CRUD operations** (Create, Read, Update, Delete) in web development and APIs.

2. Sending and receiving responses in DRF.

In **Django REST Framework (DRF)**, responses are sent and received using the **Request** and **Response** objects.

- **Receiving data:** DRF uses `request.data` to receive data sent by the client (JSON, form data, etc.).
- **Sending data:** DRF uses the `Response` class to send data back to the client in JSON format along with HTTP status codes.

DRF automatically handles content negotiation, making API communication clean, structured, and easy to manage.

1. Understanding views in DRF: Function-based views vs Class-based views.

In **Django REST Framework (DRF)**, views handle incoming HTTP requests and return responses. DRF supports two main types of views: **Function-Based Views (FBVs)** and **Class-Based Views (CBVs)**.

Function-Based Views (FBVs)

- Written as simple Python functions
- Use decorators like `@api_view`
- Easy to understand and implement
- Best for small or simple APIs

Example use: Simple CRUD operations

Class-Based Views (CBVs)

- Written as Python classes
- Provide better code organization
- Support reusable and extendable logic
- Include `APIView`, `Generic Views`, and `ViewSets`

Example use: Large and complex APIs

Key Differences

Feature	FBVs	CBVs
Structure	Function-based	Class-based
Reusability	Low	High
Complexity	Simple	More powerful
Best for	Small projects	Large projects

1. Defining URLs and linking them to views.

In Django, **URLs** define how incoming requests are mapped to specific **views**. URL configuration acts as a bridge between the user's request and the logic that handles it.

Why URL Configuration Is Important

- Directs requests to the correct view
- Organizes application routes
- Makes the project structured and maintainable

How URLs Are Defined

- URLs are defined in `urls.py`
- Each URL pattern is linked to a view function or class
- Django uses URL patterns to match incoming requests

Linking URLs to Views

- Function-based views are linked directly to URL patterns
- Class-based views are linked using `.as_view()`

Benefits

- Clear request handling
- Easy navigation and API routing
- Supports clean and readable URLs

1. Adding pagination to APIs to handle large data sets.

Pagination is used in APIs to **divide large amounts of data into smaller, manageable pages** instead of returning all records at once. This improves performance and reduces server load.

Why Pagination Is Important

- Faster API responses
- Reduced memory usage
- Better user experience
- Efficient handling of large datasets

Pagination in Django REST Framework (DRF)

DRF provides built-in pagination classes such as:

- PageNumberPagination
- LimitOffsetPagination
- CursorPagination

Pagination can be enabled globally or per API view.

Benefits

- Improves scalability
- Makes APIs more efficient
- Helps frontend load data smoothly

1. Configuring Django settings for database, static files, and API keys.

Database Configuration

Database settings in settings.py define how Django connects to the database (MySQL, PostgreSQL, SQLite, etc.). Proper configuration ensures smooth data storage and retrieval.

Static Files Configuration

Static files include CSS, JavaScript, and images. Django settings specify:

- Static file URL
- Static file directories
- Static root for deployment
This ensures static content is served correctly.

API Keys Configuration

API keys and secret values should be stored securely using:

- Environment variables
- .env files
This prevents exposing sensitive data in source code.

Project Setup

1. Setting up a Django REST Framework project.

Setting up a DRF project involves creating a Django project and integrating Django REST Framework to build APIs.

Steps to Set Up a DRF Project

1. Install Django and DRF

```
pip install django djangorestframework
```

2. Create a Django Project

```
django-admin startproject project_name
```

```
cd project_name
```

3. Create an App

```
python manage.py startapp app_name
```

4. Add Apps to INSTALLED_APPS in settings.py

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    ...  
    'rest_framework',  
    'app_name',  
]
```

5. Define Models, Serializers, and Views

- Create models in models.py
- Create serializers in serializers.py
- Create API views in views.py

6. Configure URLs

- Link API endpoints to views in urls.py using Django's path() or include() functions.

7. Run Migrations

```
python manage.py makemigrations
```

```
python manage.py migrate
```

8. Run the Development Server

```
python manage.py runserver
```

1. social authentication (e.g., Google, Facebook) in Django.

1. Social Authentication (e.g., Google, Facebook) in Django

Social authentication allows users to log in or register using their accounts from social platforms like **Google, Facebook, or GitHub**, instead of creating a new username and password. This improves user experience and simplifies authentication.

Why Social Authentication Is Useful

- Faster signup/login for users
 - Reduces password management issues
 - Enhances user trust with familiar platforms
 - Can access basic user profile information (email, name, profile picture)
-

How to Implement Social Authentication in Django

1. Install Required Packages

```
pip install django-allauth
```

2. Add Apps to INSTALLED_APPS

```
INSTALLED_APPS = [  
    'django.contrib.sites',  
    'allauth',  
    'allauth.account',  
    'allauth.socialaccount',  
    'allauth.socialaccount.providers.google',  
    'allauth.socialaccount.providers.facebook',  
    ...  
]
```

3. Configure Site ID in settings.py

```
SITE_ID = 1
```

4. Add URLs for Authentication

```
urlpatterns = [  
    path('accounts/', include('allauth.urls')),  
]
```

5. Set Up API Keys on Social Platforms

- Create OAuth credentials on Google/Facebook developer console
- Add **Client ID** and **Client Secret** to Django settings

6. Login Using Social Accounts

- Users can now log in via Google or Facebook buttons on the frontend.

2. Sending emails and OTPs using third-party APIs like Twilio, SendGrid.

Sending emails and OTPs (One-Time Passwords) is essential for **user verification, notifications, and security** in web applications. Third-party services like **Twilio** (for SMS) and **SendGrid** (for emails) make this process reliable and easy.

Why Use Third-Party APIs

- Ensures high delivery rate
- Handles large volume of messages
- Reduces development complexity
- Provides security and analytics

1. Sending Emails Using SendGrid

1. Install SendGrid Package

```
pip install sendgrid
```

2. Configure API Key in Django settings.py or environment variables.

3. Send Email Example:

```
from sendgrid import SendGridAPIClient
```

```
from sendgrid.helpers.mail import Mail
```

```
message = Mail(
```

```
    from_email='your_email@example.com',
```

```
    to_emails='user@example.com',
```

```
    subject='Welcome!',
```

```
    html_content='<strong>Thanks for registering!</strong>'
```

```
)
```

```
sg = SendGridAPIClient('YOUR_API_KEY')
```

```
response = sg.send(message)
```

2. Sending OTPs Using Twilio

1. Install Twilio Package

```
pip install twilio
```

2. Configure Account SID, Auth Token (from Twilio console).

3. Send OTP Example:

```
from twilio.rest import Client
```

```
client = Client('TWILIO_ACCOUNT_SID', 'TWILIO_AUTH_TOKEN')
```

```
message = client.messages.create(
```

```
    body='Your OTP is 123456',
```

```
    from_='+1234567890',
```

```
    to='+91XXXXXXXXXX'
```

```
)
```

1. REST principles: statelessness, resource-based URLs, and using HTTP methods for CRUD operations.

REST (Representational State Transfer) is an architectural style for designing web APIs. It follows specific principles to ensure scalability, simplicity, and efficient communication between clients and servers.

Key REST Principles

1. Statelessness

- Each API request from a client contains all the information needed to process it.
- The server **does not store client state** between requests.
- Improves scalability and reduces server complexity.

Example:

Every API request must include authentication credentials; the server does not remember previous requests.

2. Resource-Based URLs

- Resources (data objects) are identified using URLs.
- Each resource has a unique endpoint.
- Makes APIs organized and intuitive.

Example:

```
/users/      # All users
/users/1/    # Specific user with ID 1
/products/   # All products
/products/10/ # Product with ID 10
```

3. Using HTTP Methods for CRUD Operations

REST uses standard HTTP methods to perform **CRUD (Create, Read, Update, Delete)** operations:

HTTP Method	Operation	Example
GET	Read	/users/ → Retrieve list of users
POST	Create	/users/ → Create a new user
PUT	Update	/users/1/ → Update all details of user 1
PATCH	Partial Update	/users/1/ → Update some fields of user 1
DELETE	Delete	/users/1/ → Delete user 1

1. What is CRUD, and why is it fundamental to backend development?

CRUD stands for **Create, Read, Update, and Delete**—the four basic operations for managing data in a database. It is a core concept in backend development because almost every application needs to interact with data.

CRUD Operations

Operation	Description	Example
Create	Add new data to the database	Registering a new user
Read	Retrieve or view existing data	Displaying user profiles
Update	Modify existing data	Updating a user's email or password
Delete	Remove data	Deleting a user account

Why CRUD Is Fundamental

1. **Data Management:** Every backend application deals with storing, retrieving, and modifying data. CRUD defines these operations clearly.
2. **API Design:** REST APIs are often designed around CRUD operations using HTTP methods:
 - **POST** → Create
 - **GET** → Read
 - **PUT/PATCH** → Update
 - **DELETE** → Delete
3. **Consistency:** Provides a standard approach for interacting with databases and ensures predictable behavior.
4. **Scalability:** CRUD operations form the backbone for more complex features like analytics, reporting, and user management.

1. Difference between authentication and authorization.

Authentication and **Authorization** are related but serve different purposes in web applications.

Authentication

- **What it is:** Verifying the identity of a user.
- **Purpose:** Ensures the user is who they claim to be.
- **Example:** Logging in with username/password or using Google login.
- **Question it answers:** “Who are you?”

Authorization

- **What it is:** Granting access to resources or actions based on user permissions.
- **Purpose:** Controls what an authenticated user can do.
- **Example:** Only admins can delete posts; regular users can only view them.
- **Question it answers:** “What can you do?”

Key Differences

Feature	Authentication	Authorization
Purpose	Verify identity	Grant access
Occurs	First	After authentication
Example	Login	Accessing admin panel
Question	Who are you?	What are you allowed to do?

2. Implementing authentication using Django REST Framework’s token-based system.

Token-based authentication in Django REST Framework (DRF) allows clients to authenticate using a **token** instead of a username and password for every request. It is widely used for APIs because it is **secure, stateless, and scalable**.

Steps to Implement Token-Based Authentication

1. Install DRF (if not installed)

```
pip install djangorestframework
```

2. Add Apps to INSTALLED_APPS

```
INSTALLED_APPS = [  
    'rest_framework',  
    'rest_framework.authtoken',  
    'your_app',  
]
```

3. Run Migrations for Tokens

```
python manage.py migrate
```

4. Generate Tokens for Users

DRF provides a Token model. You can create tokens using Django shell or automatically when a user registers.

```
from rest_framework.authtoken.models import Token  
from django.contrib.auth.models import User
```

```
user = User.objects.get(username='john')  
token = Token.objects.create(user=user)  
print(token.key)
```

5. Configure DRF Authentication in settings.py

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        'rest_framework.authentication.TokenAuthentication',  
    ]  
}
```

6. Protect API Views Using Token Authentication

- Function-Based View Example:

```
from rest_framework.decorators import api_view, authentication_classes, permission_classes  
from rest_framework.authentication import TokenAuthentication  
from rest_framework.permissions import IsAuthenticated  
from rest_framework.response import Response
```

```
@api_view(['GET'])
```

```
@authentication_classes([TokenAuthentication])
@permission_classes([IsAuthenticated])
def protected_view(request):
    return Response({"message": f"Hello {request.user.username}"})
```

How It Works

1. User logs in with username/password.
2. Server returns a **token**.
3. Client includes the token in the **Authorization header** for subsequent requests:
Authorization: Token <user_token>
4. Server validates the token and allows access if valid.

1. Introduction to OpenWeatherMap API and how to retrieve weather data.

OpenWeatherMap (OWM) is a popular **third-party API** that provides real-time weather data, forecasts, and historical weather information for locations worldwide. It is commonly used in web and mobile applications to display weather information.

Key Features

- Current weather data (temperature, humidity, wind speed, etc.)
- 5-day / 7-day forecasts
- Historical weather data
- Support for cities, coordinates, and ZIP codes

How to Use OpenWeatherMap API

1. Get an API Key

- Sign up at [OpenWeatherMap](https://openweathermap.org/)
- Generate a free API key from the dashboard

2. Construct API Request

- Base URL for current weather:
`http://api.openweathermap.org/data/2.5/weather?q={city_name}&appid={API_key}&units=metric`
- Replace {city_name} with the city and {API_key} with your key
- units=metric returns temperature in Celsius

3. Make API Request

You can use Python's requests library to fetch data:

```
import requests
```

```
API_KEY = 'your_api_key'
```

```
city = 'Ahmedabad'
```

```
url =
```

```
f'http://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metric'
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
print(data)
```

4. Access Weather Data

The API returns JSON data. Example fields:

- `data['main']['temp']` → Temperature
- `data['weather'][0]['description']` → Weather description
- `data['wind']['speed']` → Wind speed

1. Using Google Maps Geocoding API to convert addresses into coordinates.

The **Google Maps Geocoding API** allows developers to convert a physical **address** into geographic **coordinates (latitude and longitude)**, and vice versa. This is essential for mapping, location-based services, and GIS applications.

Key Features

- Convert addresses into coordinates (geocoding)
- Convert coordinates into addresses (reverse geocoding)
- Supports global locations
- Returns results in JSON or XML

How to Use Google Maps Geocoding API

1. Get an API Key

- Sign up for the [Google Cloud Platform](#)
- Enable **Geocoding API** in the console
- Generate an API key

2. Construct the API Request

- Base URL for geocoding:
`https://maps.googleapis.com/maps/api/geocode/json?address={ADDRESS}&key={API_KEY}`
- Replace {ADDRESS} with the location and {API_KEY} with your key

3. Make API Request in Python

```
import requests
```

```
API_KEY = 'your_api_key'
```

```
address = 'Ahmedabad, India'
```

```
url =
```

```
f'https://maps.googleapis.com/maps/api/geocode/json?address={address}&key={API_KEY}'
```

```
response = requests.get(url)
```



```
data = response.json()

# Extract coordinates
if data['status'] == 'OK':
    location = data['results'][0]['geometry']['location']
    latitude = location['lat']
    longitude = location['lng']
    print(f"Latitude: {latitude}, Longitude: {longitude}")
else:
    print("Error:", data['status'])
```

4. Output

The API returns JSON containing:

- lat → Latitude
- lng → Longitude
- Full address and additional details

Advantages

- Accurate global geocoding
- Easy integration with Google Maps
- Supports batch and reverse geocoding
- JSON format is easy to use in web and mobile apps

1. Introduction to GitHub API and how to interact with repositories, pull requests, and issues.

The **GitHub API** allows developers to interact programmatically with GitHub's features, such as repositories, pull requests, and issues. It enables automation, integration with other tools, and data retrieval for web or desktop applications.

Key Features of GitHub API

- Access and manage **repositories** (create, read, update, delete)
- Work with **pull requests** (create, review, merge)
- Manage **issues** (open, comment, close)
- Retrieve user and organization data
- Supports **REST API** and **GraphQL API**

How to Interact with GitHub API

1. Get a Personal Access Token

- Go to [GitHub Settings → Developer Settings → Personal Access Tokens]
- Generate a token with required scopes (repo, workflow, admin:org, etc.)

2. Make API Requests

- GitHub API uses **HTTPS requests** and returns JSON data
- Include your **token** in the header for authentication

Example: List Repositories for a User

```
import requests
```

```
token = "your_personal_access_token"
```

```
headers = {"Authorization": f"token {token}"}
```

```
username = "octocat"
```

```
url = f"https://api.github.com/users/{username}/repos"
```

```
response = requests.get(url, headers=headers)
```

```
repos = response.json()
```

```
for repo in repos:
```

```
print(repo['name'], repo['html_url'])
```

3. Interacting with Pull Requests

- Get PRs for a repository:

```
url = f"https://api.github.com/repos/{username}/{repo_name}/pulls"
```

```
response = requests.get(url, headers=headers)
```

```
pulls = response.json()
```

- You can also **create, merge, or comment** on pull requests using POST/PUT methods.

4. Working with Issues

- List issues:

```
url = f"https://api.github.com/repos/{username}/{repo_name}/issues"
```

```
response = requests.get(url, headers=headers)
```

```
issues = response.json()
```

- Create new issues using POST requests with a JSON body containing title and body.

Advantages

- Automate repository management
- Track issues and pull requests programmatically
- Integrate GitHub with CI/CD pipelines or applications
- Retrieve real-time data for dashboards or analytics

1. Using Twitter API to fetch and post tweets, and retrieve user data.

The **Twitter API** allows developers to interact programmatically with Twitter, enabling tasks like fetching tweets, posting new tweets, and accessing user profile information. It is widely used for building apps, analytics, and automation.

Key Features

- **Fetch Tweets:** Retrieve tweets from timelines, hashtags, or specific users
- **Post Tweets:** Publish new tweets from your application
- **User Data:** Access profile info like followers, following, and account stats
- **Search & Streams:** Filter tweets in real-time or by keywords

How to Use the Twitter API

1. Get API Credentials

- Sign up at [Twitter Developer Portal](#)
- Create a project and app
- Generate the following credentials:
 - API Key & Secret
 - Access Token & Secret
 - Bearer Token (for v2 API)

2. Install Python Library

```
pip install tweepy
```

Tweepy simplifies Twitter API integration.

3. Authenticate with Twitter

```
import tweepy
```

```
# Credentials
```

```
api_key = "YOUR_API_KEY"
```

```
api_key_secret = "YOUR_API_SECRET"
```

```
access_token = "YOUR_ACCESS_TOKEN"
```

```
access_token_secret = "YOUR_ACCESS_TOKEN_SECRET"
```

```
# Authentication
```

```
auth = tweepy.OAuth1UserHandler(api_key, api_key_secret, access_token,  
access_token_secret)
```

```
api = tweepy.API(auth)
```

4. Fetch Tweets

```
tweets = api.user_timeline(screen_name="twitter_user", count=5)
```

```
for tweet in tweets:
```

```
    print(tweet.text)
```

5. Post a Tweet

```
api.update_status("Hello world! This is a tweet from my Python app.")
```

6. Retrieve User Data

```
user = api.get_user(screen_name="twitter_user")
```

```
print("Username:", user.screen_name)
```

```
print("Followers:", user.followers_count)
```

```
print("Following:", user.friends_count)
```

Advantages

- Automate posting tweets
- Analyze trends and user behavior
- Build Twitter-based applications or dashboards
- Access real-time data for analytics

1. Introduction to REST Countries API and how to retrieve country-specific data.

The **REST Countries API** is a free, public API that provides detailed information about countries worldwide. It is commonly used in web and mobile applications to display **country data** such as population, capital, region, currencies, languages, and more.

Key Features

- Retrieve information about **all countries** or a specific country
- Access data like:
 - Name, capital, region, and subregion
 - Population and area
 - Currencies and languages
 - Timezones and borders
- Supports JSON responses, making it easy to use in apps

How to Use REST Countries API

1. Base URL

<https://restcountries.com/v3.1/>

2. Fetch All Countries

```
import requests
```

```
url = "https://restcountries.com/v3.1/all"
```

```
response = requests.get(url)
```

```
countries = response.json()
```

```
for country in countries[:5]: # Display first 5 countries
```

```
    print(country['name']['common'], "-", country['capital'])
```

3. Fetch Data for a Specific Country

```
country_name = "India"
```

```
url = f"https://restcountries.com/v3.1/name/{country_name}"
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
print("Country:", data[0]['name']['common'])
```

```
print("Capital:", data[0]['capital'][0])
```

```
print("Region:", data[0]['region'])
```

```
print("Population:", data[0]['population'])
```

4. Fetch Country by Code

```
country_code = "IN" # ISO 3166-1 alpha-2 code
```

```
url = f"https://restcountries.com/v3.1/alpha/{country_code}"
```

```
response = requests.get(url)
```

```
data = response.json()
```

```
print(data[0]['name']['common'], "-", data[0]['capital'][0])
```

Advantages

- Free and publicly accessible
- Easy JSON format integration
- Provides extensive country information
- Supports filtering by region, currency, language, and code

1. Using email sending APIs like SendGrid and Mailchimp to send transactional emails.

Email sending APIs allow applications to send automated emails, such as registration confirmations, password resets, and notifications. Services like **SendGrid** and **Mailchimp** provide reliable and scalable solutions for sending **transactional emails**.

Why Use Email Sending APIs

- Ensure reliable delivery of emails
- Avoid server email configuration issues
- Track email delivery, opens, and clicks
- Handle large volumes of emails efficiently

1. Sending Emails with SendGrid

Step 1: Install SendGrid

```
pip install sendgrid
```

Step 2: Configure API Key

- Sign up on [SendGrid](#)
- Generate an API key and store it securely (e.g., environment variables)

Step 3: Send Email

```
from sendgrid import SendGridAPIClient
```

```
from sendgrid.helpers.mail import Mail
```

```
message = Mail(
    from_email='you@example.com',
    to_emails='user@example.com',
    subject='Welcome!',
    html_content='<strong>Thank you for signing up!</strong>'
)

sg = SendGridAPIClient('YOUR_SENDGRID_API_KEY')
response = sg.send(message)
print(response.status_code)
```


2. Sending Emails with Mailchimp Transactional API (Mandrill)

Step 1: Install Mailchimp Package

```
pip install mailchimp-marketing
```

Step 2: Configure API Key

- Create an account on [Mailchimp](#)
- Generate **API key** under Transactional (Mandrill)

Step 3: Send Email

```
from mailchimp_transactional import Client
```

```
client = Client()
```

```
client.set_config({"api_key": "YOUR_API_KEY"})
```

```
message = {
```

```
    "from_email": "you@example.com",
```

```
    "subject": "Welcome!",
```

```
    "text": "Thanks for registering!",
```

```
    "to": [{"email": "user@example.com", "type": "to"}]
```

```
}
```

```
response = client.messages.send({"message": message})
```

```
print(response)
```

Advantages

- Fast and reliable email delivery
- Handles transactional emails securely
- Supports analytics and tracking
- Reduces workload on your own email server

1. Introduction to Twilio API for sending SMS and OTPs.

Twilio API is a cloud-based service that allows developers to **send SMS messages, voice calls, and OTPs** programmatically. It is widely used in web and mobile applications for **user verification, notifications, and two-factor authentication (2FA)**.

Key Features

- Send **SMS messages** globally
- Send **One-Time Passwords (OTPs)** for verification
- Make and receive **voice calls**
- Supports **WhatsApp messaging**
- Provides delivery and status tracking

Why Use Twilio

- Reliable and fast message delivery
- Scalable for large volumes
- Easy integration with applications
- Secure method for user authentication

How to Use Twilio API

1. Create a Twilio Account

- Sign up at [Twilio](https://www.twilio.com)
- Get **Account SID**, **Auth Token**, and a Twilio phone number

2. Install Twilio Python Library

```
pip install twilio
```

3. Send SMS / OTP

```
from twilio.rest import Client
```

```
# Twilio credentials
```

```
account_sid = 'YOUR_ACCOUNT_SID'
```

```
auth_token = 'YOUR_AUTH_TOKEN'
```

```
client = Client(account_sid, auth_token)
```

Send SMS

```
message = client.messages.create(  
    body="Your OTP is 123456",  
    from_='+1234567890', # Twilio number  
    to='+919876543210' # Recipient number  
)
```

```
print("Message SID:", message.sid)
```

Advantages

- Easy to send SMS and OTPs programmatically
- Secure for user verification
- Provides delivery reports
- Supports multiple messaging channels

1. Introduction to integrating payment gateways like PayPal and Stripe.

Payment gateways allow web and mobile applications to process online payments securely. Popular gateways like **PayPal** and **Stripe** enable businesses to accept payments from users using credit cards, debit cards, wallets, and other methods.

Key Features of Payment Gateways

- Secure processing of online payments
- Support for multiple payment methods (cards, wallets, UPI, etc.)
- Fraud detection and compliance with security standards (PCI DSS)
- Transaction tracking, refunds, and recurring payments

Why Integrate Payment Gateways

- Enables e-commerce and subscription-based services
- Simplifies payment process for users
- Ensures safe and reliable transactions
- Reduces the burden of handling sensitive financial data

How to Integrate PayPal

1. **Sign up for a PayPal Business Account**
2. **Get API credentials** (Client ID and Secret)
3. **Use PayPal SDKs** or REST API to create payment buttons or process payments

Basic Flow:

- User clicks “Pay with PayPal”
- Redirect to PayPal for payment
- Return to website with payment status

How to Integrate Stripe

1. **Create a Stripe Account**
2. **Get API keys** (Publishable key and Secret key)
3. **Use Stripe SDK or API** to process payments directly on the website/app

Basic Flow:

- Collect card details securely using Stripe Elements
- Create a payment intent
- Confirm the payment and handle the response

Advantages

- Secure and PCI-compliant transactions
- Supports global payments
- Provides dashboards for transaction management
- Easy integration with web and mobile applications

1. Using Google Maps API to display maps and calculate distances between locations.

The **Google Maps API** allows developers to embed interactive maps in web and mobile applications and perform location-based operations such as calculating distances, routes, and directions.

Key Features

- Display interactive maps with markers
- Calculate **distance and travel time** between locations
- Get directions for driving, walking, cycling, or transit
- Geocoding and reverse geocoding (convert addresses ↔ coordinates)
- Supports street view, traffic layers, and custom overlays

How to Use Google Maps API

1. Get an API Key

- Sign up at [Google Cloud Platform](https://cloud.google.com/maps-platform/)
- Enable **Maps JavaScript API** and **Distance Matrix API**
- Generate an API key

2. Display a Map on a Web Page

```
<!DOCTYPE html>

<html>

<head>

  <title>Simple Map</title>

  <script src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY"></script>

  <script>

    function initMap() {

      var location = { lat: 23.0225, lng: 72.5714 }; // Ahmedabad

      var map = new google.maps.Map(document.getElementById('map'), {

        zoom: 10,

        center: location

      });
```

```

        var marker = new google.maps.Marker({ position: location, map: map });
    }
</script>
</head>
<body onload="initMap()">
    <div id="map" style="height:500px;width:100%;"></div>
</body>
</html>

```

3. Calculate Distance Between Two Locations

Using the **Distance Matrix API**:

```
import requests
```

```
API_KEY = 'YOUR_API_KEY'
```

```
origin = "Ahmedabad, India"
```

```
destination = "Surat, India"
```

```

url =
f"https://maps.googleapis.com/maps/api/distancematrix/json?origins={origin}&destinations={
destination}&key={API_KEY}"
response = requests.get(url).json()

```

```
distance = response['rows'][0]['elements'][0]['distance']['text']
```

```
duration = response['rows'][0]['elements'][0]['duration']['text']
```

```
print(f"Distance: {distance}, Duration: {duration}")
```

Advantages

- Interactive and customizable maps
- Accurate distance and route calculation
- Supports multiple travel modes
- Easy integration with web and mobile apps

